

[CI2] Concepts informatiques 2 — Cours 1

Jérémy Ledent <jeremy.ledent@irif.fr>

Université Paris Cité, IRIF



INSTITUT
DE RECHERCHE
EN INFORMATIQUE
FONDAMENTALE



Université
Paris Cité

Organisation du cours

- ▶ La page Moodle du cours :

<https://moodle.u-paris.fr/course/view.php?id=1625>

contient les transparents du cours, les feuilles de TD, les dates d'examen.

- ▶ Les TD commenceront la semaine du **27 janvier**.

Inscription aux groupes de TD

[Cours](#) [Paramètres](#) [Participants](#) [Notes](#) [Rapports](#) [Plus ▾](#)

 **IFA2E025 Concepts Informatiques 2**

 **Présentation**

Tout replier

Ceci est la page Moodle pour le cours [CI2] Concepts Informatiques.

 [Annonces](#)

 [Inscription à un groupe de TD](#)

 **Séances et intervenants**

 **Inscrivez-vous !**

Modalités de contrôle des connaissances

- ▶ Semaine du TD 4 : Contrôle TD n°1
- ▶ Semaine du TD 7 : Partiel
- ▶ Semaine du TD 11 : Contrôle TD n°2
- ▶ Examen final

Note de la première session : $\max(30\% \text{ CC} + 20\% \text{ Partiel} + 50\% \text{ Exam}, \text{Exam})$

Note de la session seconde chance : $\max(30\% \text{ CC} + 20\% \text{ Partiel} + 50\% \text{ SC}, \text{SC})$

Contenu du cours

Résumé des épisodes précédents

Au semestre 1, vous avez vu (entre autres) :

- ▶ **CI1** : comment encoder de l'information dans un ordinateur, et comment des circuits électroniques composés de portes logiques permettent d'effectuer des calculs.
- ▶ **IP1** : un langage de programmation impérative (Java) et ses concepts fondamentaux : types de données, conditions, boucles, tableaux.

Résumé des épisodes précédents

Au semestre 1, vous avez vu (entre autres) :

- ▶ **CI1** : comment encoder de l'information dans un ordinateur, et comment des circuits électroniques composés de portes logiques permettent d'effectuer des calculs.
- ▶ **IP1** : un langage de programmation impérative (Java) et ses concepts fondamentaux : types de données, conditions, boucles, tableaux.

Objectif CI2 :

Comment passe-t-on des instructions machine bas niveau traitées par le processeur, à un langage de programmation haut niveau comme Java ?

► Partie 1 :

- Les **pires** comme structure de donnée.
- Organisation de la mémoire : la **pile** et le **tas**.
- Où sont stockées en mémoire les données d'un programme : variables locales et globales, adresses, références, fonctions, paramètres, tableaux, objets.

► Partie 1 :

- Les **pires** comme structure de donnée.
- Organisation de la mémoire : la **pile** et le **tas**.
- Où sont stockées en mémoire les données d'un programme : variables locales et globales, adresses, références, fonctions, paramètres, tableaux, objets.

► Partie 2 :

- Introduction au langage machine : à quoi ressemble un **langage assembleur**?
- Traduction (**compilation**) de programmes Java vers un langage assembleur.

► Partie 1 :

- Les **pires** comme structure de donnée.
- Organisation de la mémoire : la **pile** et le **tas**.
- Où sont stockées en mémoire les données d'un programme : variables locales et globales, adresses, références, fonctions, paramètres, tableaux, objets.

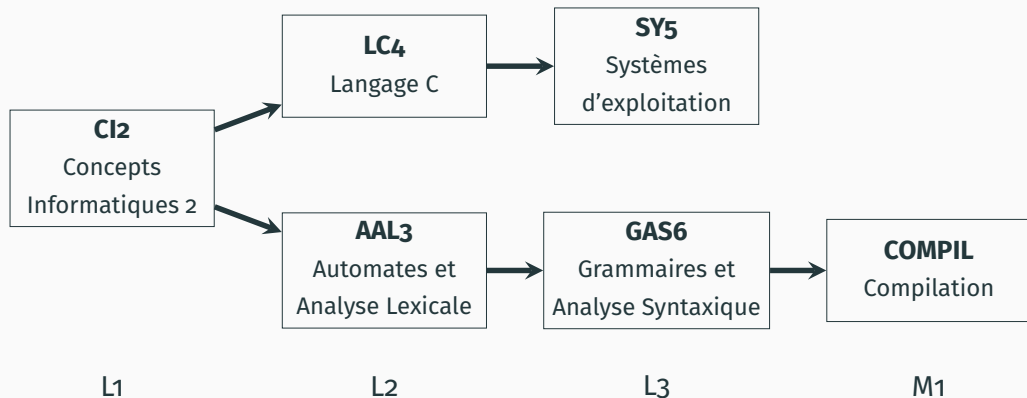
► Partie 2 :

- Introduction au langage machine : à quoi ressemble un **langage assembleur**?
- Traduction (**compilation**) de programmes Java vers un langage assembleur.

► Partie 3 :

- Aspects bas niveau de la **récursivité**.
- Récursivité terminale, mémoïsation.
- Backtracking.

Vers où on va ?



Un mot sur les amphis et les TDs

CI2 = **Concepts** Informatiques $\implies$ C'est un cours **théorique**.

On y acquiert des **connaissances**, pas (encore) des compétences.

Un mot sur les amphis et les TDs

CI2 = **Concepts** Informatiques $\implies$ C'est un cours **théorique**.

On y acquiert des **connaissances**, pas (encore) des compétences.

Chaque semaine :

- ▶ 2h CM $\longleftarrow$ les notions à retenir sont ici!
- ▶ 2h TD $\longleftarrow$ on met en pratique ce qui a été vu en CM.

Un mot sur les amphis et les TDs

CI2 = **Concepts** Informatiques $\implies$ C'est un cours **théorique**.

On y acquiert des **connaissances**, pas (encore) des compétences.

Chaque semaine :

- ▶ 2h CM $\longleftarrow$ les notions à retenir sont ici!
- ▶ 2h TD $\longleftarrow$ on met en pratique ce qui a été vu en CM.

Les questions du type :

« Ça sert à quoi ce qu'on est en train de faire là ? »

sont **bienvenues** ! Ne bachotez pas les exercices de TD sans comprendre leur but.

La pile, le tas

Quiz!

<https://app.wooclap.com/QUIZCI2>

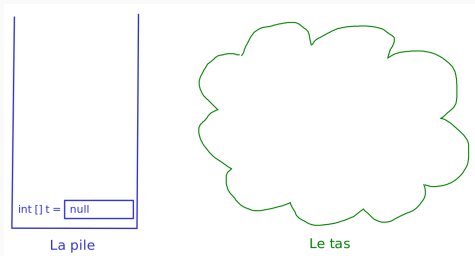
Organisation de la mémoire : la pile et le tas

Lors de l'exécution d'un programme, la mémoire est divisée en deux zones :

- ▶ **La pile** (en anglais, **stack**) : c'est là que sont stockées les **variables locales** et les **paramètres** de chaque fonction du programme. Ces données sont organisées de manière structurée (cf. structure de pile LIFO, on y vient).
- ▶ **Le tas** (en anglais, **heap**) : c'est là que sont stockées les données du programme (**tableaux**, **objets**). Ces données sont *partagées* entre les différentes fonctions. Elles sont stockées « en vrac », d'où le nom de “tas”.

Exemple 1 : Tableau d'entiers

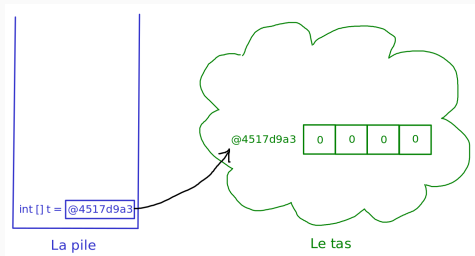
```
► int[] t;  
  t = new int[4];  
  for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
  }
```



- La variable `t` est une **référence** vers un tableau d'entiers.
- La variable `t` est déclarée, mais aucun tableau d'entiers n'est encore créé!
- Pour l'instant, `t` contient une référence indéfinie : `null`.

Exemple 1 : Tableau d'entiers

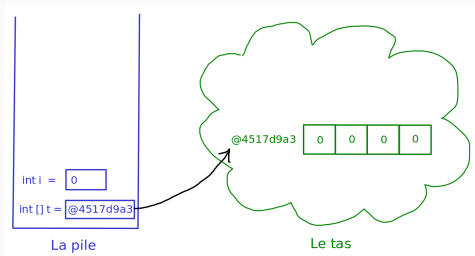
```
int[] t;  
▶ t = new int[4];  
for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
}
```



- ▶ Le mot clé `new` permet d'allouer de la mémoire dans le tas.
- ▶ Java réserve la mémoire nécessaire pour stocker 4 entiers, et retient l'adresse mémoire correspondante dans la variable `t`.
- ▶ La valeur par défaut du type `int` est 0.

Exemple 1 : Tableau d'entiers

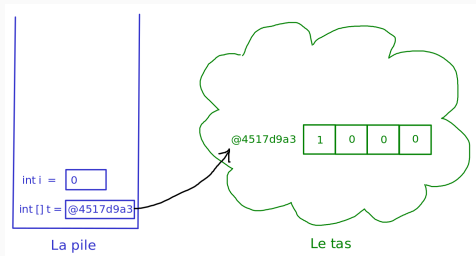
```
int[] t;  
t = new int[4];  
▶ for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
}
```



- ▶ La variable `i` est une variable **locale**.
- ▶ Elle est stockée dans la pile.

Exemple 1 : Tableau d'entiers

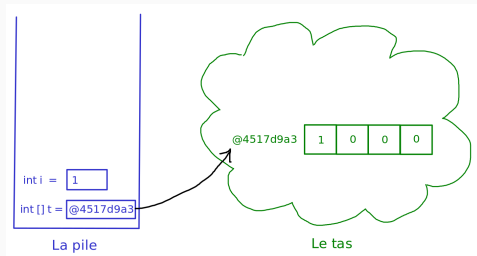
```
int[] t;  
t = new int[4];  
for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
}
```



- L'expression `t[i]` permet d'accéder à la i -ème case du tableau référencé par la variable `t`.

Exemple 1 : Tableau d'entiers

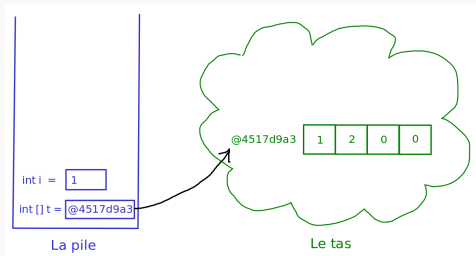
```
int[] t;  
t = new int[4];  
▶ for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
}
```



- ▶ On continue de remplir le tableau `t` jusqu'à obtenir `{1, 2, 3, 4}`.

Exemple 1 : Tableau d'entiers

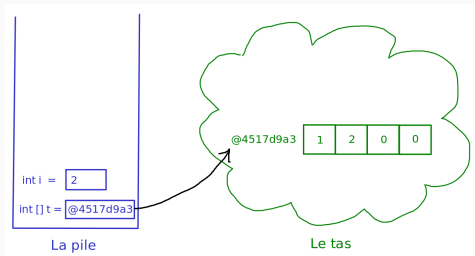
```
int[] t;  
t = new int[4];  
for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
}
```



- On continue de remplir le tableau `t` jusqu'à obtenir `{1, 2, 3, 4}`.

Exemple 1 : Tableau d'entiers

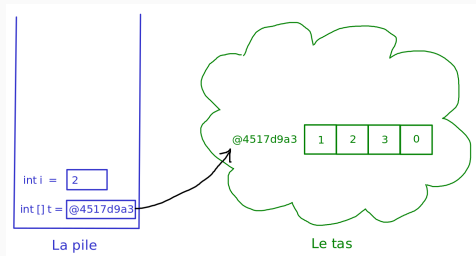
```
int[] t;  
t = new int[4];  
▶ for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
}
```



- ▶ On continue de remplir le tableau `t` jusqu'à obtenir `{1, 2, 3, 4}`.

Exemple 1 : Tableau d'entiers

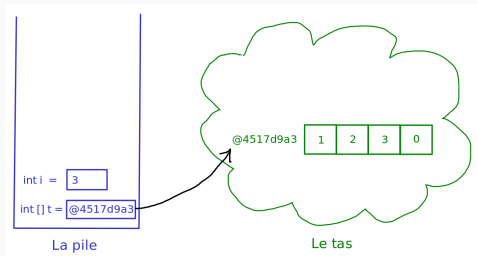
```
int[] t;  
t = new int[4];  
for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
}
```



- On continue de remplir le tableau `t` jusqu'à obtenir `{1, 2, 3, 4}`.

Exemple 1 : Tableau d'entiers

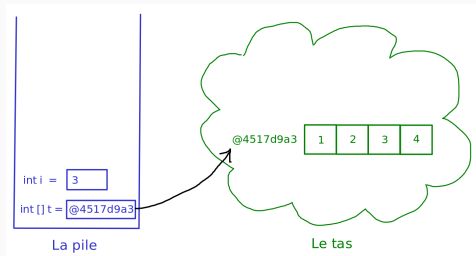
```
int[] t;  
t = new int[4];  
▶ for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
}
```



- ▶ On continue de remplir le tableau `t` jusqu'à obtenir `{1, 2, 3, 4}`.

Exemple 1 : Tableau d'entiers

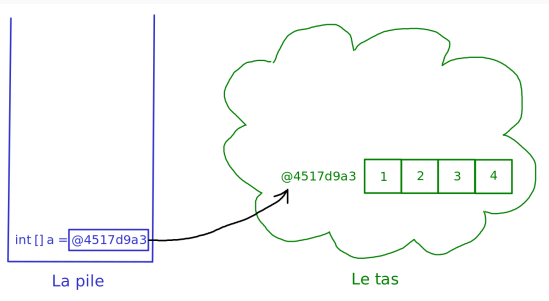
```
int[] t;  
t = new int[4];  
for (int i = 0; i < t.length; i++) {  
    t[i] = i + 1;  
}
```



- On continue de remplir le tableau `t` jusqu'à obtenir `{1, 2, 3, 4}`.

Exemple 2 : Copie et comparaison de tableaux

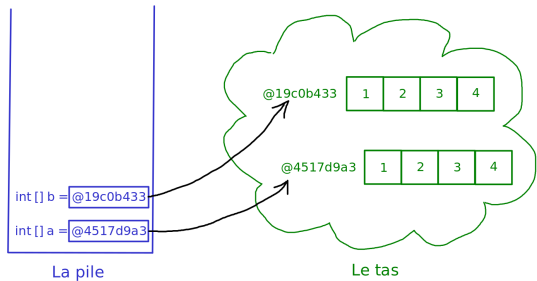
```
► int[] a = {1, 2, 3, 4};  
int[] b = {1, 2, 3, 4};  
System.out.println(a == b);  
int[] c = a;  
a[0] = 47;  
System.out.println(c[0]);  
System.out.println(a == c);
```



On alloue un tableau {1, 2, 3, 4} dans le tas, et on stocke son adresse mémoire dans la variable `a`.

Exemple 2 : Copie et comparaison de tableaux

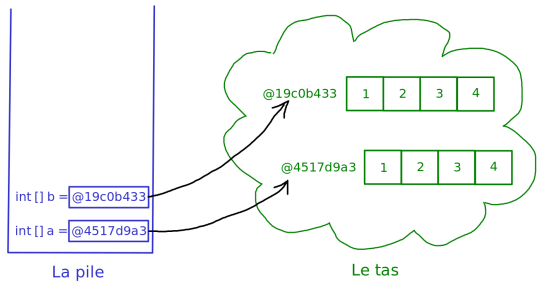
```
int[] a = {1, 2, 3, 4};  
▶ int[] b = {1, 2, 3, 4};  
System.out.println(a == b);  
int[] c = a;  
a[0] = 47;  
System.out.println(c[0]);  
System.out.println(a == c);
```



On alloue un **autre** tableau {1, 2, 3, 4} dans le tas, et on stocke son adresse mémoire dans la variable `b`.

Exemple 2 : Copie et comparaison de tableaux

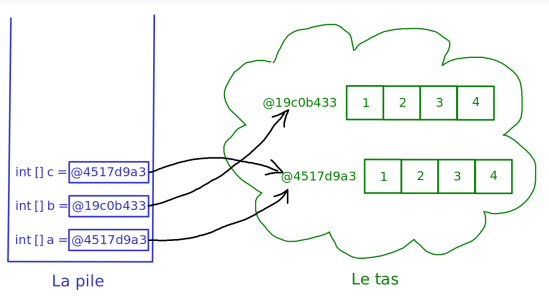
```
int[] a = {1, 2, 3, 4};  
int[] b = {1, 2, 3, 4};  
► System.out.println(a == b);  
int[] c = a;  
a[0] = 47;  
System.out.println(c[0]);  
System.out.println(a == c);
```



L'opérateur == compare les **addresses mémoires** des deux tableaux, pas leur contenu : on affiche **false**.

Exemple 2 : Copie et comparaison de tableaux

```
int[] a = {1, 2, 3, 4};  
int[] b = {1, 2, 3, 4};  
System.out.println(a == b);  
► int[] c = a;  
a[0] = 47;  
System.out.println(c[0]);  
System.out.println(a == c);
```

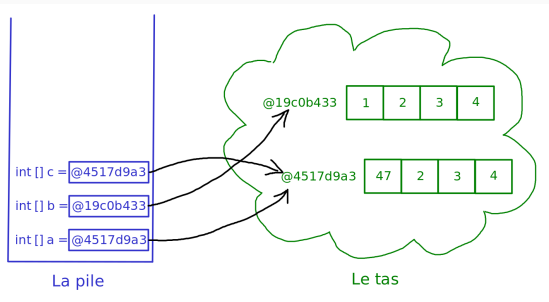


La variable `c` est une copie de la référence `a`.

On ne crée pas un nouveau tableau : on crée une deuxième référence vers un tableau existant.

Exemple 2 : Copie et comparaison de tableaux

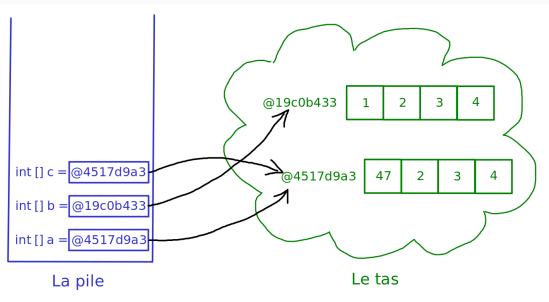
```
int[] a = {1, 2, 3, 4};  
int[] b = {1, 2, 3, 4};  
System.out.println(a == b);  
int[] c = a;  
▶ a[0] = 47;  
System.out.println(c[0]);  
System.out.println(a == c);
```



On écrit la valeur 47 dans la case numéro 0 du tableau référencé par la variable a.

Exemple 2 : Copie et comparaison de tableaux

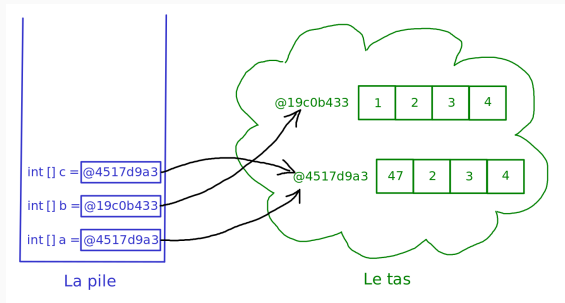
```
int[] a = {1, 2, 3, 4};  
int[] b = {1, 2, 3, 4};  
System.out.println(a == b);  
int[] c = a;  
a[0] = 47;  
▶ System.out.println(c[0]);  
System.out.println(a == c);
```



La case numéro 0 du tableau référencé par la variable `c` contient l'entier 47.

Exemple 2 : Copie et comparaison de tableaux

```
int[] a = {1, 2, 3, 4};  
int[] b = {1, 2, 3, 4};  
System.out.println(a == b);  
int[] c = a;  
a[0] = 47;  
System.out.println(c[0]);  
▶ System.out.println(a == c);
```



Les variables `a` et `c` contiennent la même adresse mémoire : on affiche `true`.

Types primitifs et Types références

Types de variables

En Java, lorsqu'on **déclare** une variable, on doit préciser son **type**.

Exemple :

```
int n = 14;
```

Types de variables

En Java, lorsqu'on **déclare** une variable, on doit préciser son **type**.

Exemple :

```
int n = 14;
```

Le **type** (dans cet exemple `int`) permet au compilateur de

- ▶ déterminer l'**espace de stockage** nécessaire pour stocker la variable
- ▶ savoir comment interpréter le contenu de cet espace.
Par exemple, un même mot binaire de 32 bits peut représenter soit un nombre entier (`int`) ou un nombre flottant (`float`).
- ▶ déterminer quelles sont les opérations qui sont autorisées pour manipuler la variable associée (**erreurs de typage**).

Variables de type primitif

Types primitifs : Les **types primitifs** représentent des données de taille fixée (8, 16, 32 ou 64 bits). On les reconnaît car ils commencent par une **lettre minuscule**.

Il en existe 8, vous les connaissez déjà presque tous :

byte

float

char

boolean

short

double

int

long

Variables de type primitif

Types primitifs : Les **types primitifs** représentent des données de taille fixée (8, 16, 32 ou 64 bits). On les reconnaît car ils commencent par une **lettre minuscule**.

Il en existe 8, vous les connaissez déjà presque tous :

byte	float	char	boolean
short	double		
int			
long			

Dans la documentation officielle de Java :

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>

Variables de types référence (Objets)

Types référence : les **types référence** (ou **types objets**) représentent des données plus complexes, qui peuvent occuper beaucoup d'espace mémoire. Leur nom commence (en général*) par une **lettre Majuscule**.

La valeur par défaut de tous les types référence est **null**.

Exemple :

```
String str = "toto";
```

Variables de types référence (Objets)

Types référence : les **types référence** (ou **types objets**) représentent des données plus complexes, qui peuvent occuper beaucoup d'espace mémoire. Leur nom commence (en général*) par une **lettre Majuscule**.

La valeur par défaut de tous les types référence est `null`.

Exemple :

```
String str = "toto";
```

* *Seule exception* : Les **types tableaux** (`int []`, `char []`, `int [] []`, `String []`, etc.) sont aussi des objets, bien qu'un peu spéciaux.

Les types référence en mémoire

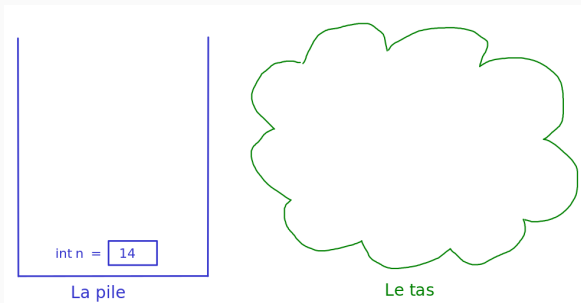
- ▶ Les variables de types primitifs sont stockées directement dans la pile.
- ▶ Pour une variable de type référence,
 - Les données (l'objet lui-même) sont stockées dans le tas.
 - Une variable contenant une référence vers cet objet est stockée dans la pile.

Les types référence en mémoire

- ▶ Les variables de types primitifs sont stockées directement **dans la pile**.
- ▶ Pour une variable de type référence,
 - Les données (l'objet lui-même) sont stockées **dans le tas**.
 - Une variable contenant une **référence** vers cet objet est stockée **dans la pile**.

Exemple :

```
▶ int n = 14;  
String str = "toto";
```

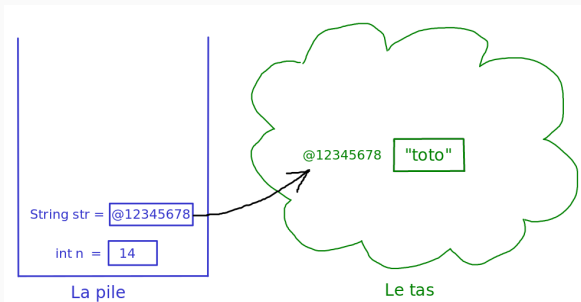


Les types référence en mémoire

- ▶ Les variables de types primitifs sont stockées directement **dans la pile**.
- ▶ Pour une variable de type référence,
 - Les données (l'objet lui-même) sont stockées **dans le tas**.
 - Une variable contenant une **référence** vers cet objet est stockée **dans la pile**.

Exemple :

```
int n = 14;  
▶ String str = "toto";
```



Types enveloppe

En Java, on a parfois besoin d'utiliser un type primitif comme si c'était un objet.

Types enveloppe

En Java, on a parfois besoin d'utiliser un type primitif comme si c'était un objet.

Type enveloppe : Chaque type primitif possède un type référence associé, appelé **type enveloppe** (ou **type boîte**, ou en anglais **wrapper**) :

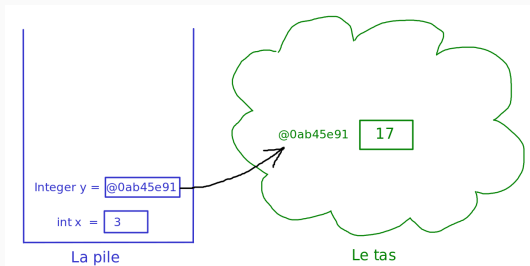
byte	short	int	long	float	double	char	boolean
Byte	Short	Integer	Long	Float	Double	Character	Boolean

Types enveloppe

En Java, on a parfois besoin d'utiliser un type primitif comme si c'était un objet.

Type enveloppe : Chaque type primitif possède un type référence associé, appelé **type enveloppe** (ou **type boîte**, ou en anglais **wrapper**) :

byte	short	int	long	float	double	char	boolean
Byte	Short	Integer	Long	Float	Double	Character	Boolean



Piles : structure de données

Piles

Une **pile** est une structure de données qui permet de stocker des éléments. Elle met en œuvre le principe du **dernier entré, premier sorti** (en anglais : **LIFO**, pour *Last In, First Out*).



Piles

Une **pile** est une structure de données qui permet de stocker des éléments. Elle met en œuvre le principe du **dernier entré, premier sorti** (en anglais : **LIFO**, pour *Last In, First Out*).



Une pile p peut être manipulée grâce à trois opérations :

Piles

Une **pile** est une structure de données qui permet de stocker des éléments. Elle met en œuvre le principe du **dernier entré, premier sorti** (en anglais : **LIFO**, pour *Last In, First Out*).



Une pile p peut être manipulée grâce à trois opérations :

- `p.empty()` teste si la pile p est vide

Une **pile** est une structure de données qui permet de stocker des éléments. Elle met en œuvre le principe du **dernier entré, premier sorti** (en anglais : **LIFO**, pour *Last In, First Out*).



Une pile p peut être manipulée grâce à trois opérations :

- `p.empty()` teste si la pile p est vide
- `p.push(x)` ajoute un élément x au sommet de la pile p . On dit qu'on a **empilé** l'élément x .

Une **pile** est une structure de données qui permet de stocker des éléments. Elle met en œuvre le principe du **dernier entré, premier sorti** (en anglais : **LIFO**, pour *Last In, First Out*).



Une pile p peut être manipulée grâce à trois opérations :

- **`p.empty()`** teste si la pile p est vide
- **`p.push(x)`** ajoute un élément x au sommet de la pile p . On dit qu'on a **empilé** l'élément x .
- **`p.pop()`** enlève la valeur au sommet de la pile et la renvoie. On dit qu'on a **dépilé** un élément.

En Java, le type des piles permettant de stocker des éléments de type T se nomme `Stack<T>`.

Attention : le type T doit être un type **référence** !

✓ `Stack<Integer>`

✗ `Stack<int>`

N'hésitez pas à jeter un coup d'œil à la documentation :

<https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html>

Exemple

```
public static void main (String[] args) {  
    ► Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =



Affichage :

>

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    ► p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =

A vertical rectangle representing a stack. The number '1' is at the bottom, and the rest of the rectangle is empty, indicating the stack contains only the value 1.

1

Affichage :

>

Example

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    ► p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =



Affichage :

>

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    ► int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =



x = 2

Affichage :

>

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    ► System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =



x = 2

Affichage :

> 2

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    ► p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =



x = 2

Affichage :

> 2

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    ► p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =



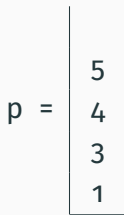
x = 2

Affichage :

> 2

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    ► p.push(5);  
    while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```



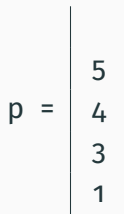
x = 2

Affichage :

> 2

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    ► while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```



x = 2

Affichage :

> 2

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        ▶ System.out.print(p.pop());  
    }  
}
```

p =



x = 2

Affichage :

> 2 5

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    ► while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =



x = 2

Affichage :

> 2 5

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        ▶ System.out.print(p.pop());  
    }  
}
```

p =



x = 2

Affichage :

> 2 5 4

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    ► while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =



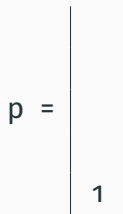
x = 2

Affichage :

> 2 5 4

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        ▶ System.out.print(p.pop());  
    }  
}
```



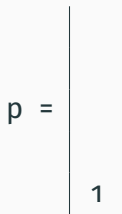
x = 2

Affichage :

> 2 5 4 3

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    ► while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```



x = 2

Affichage :

> 2 5 4 3

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        ▶ System.out.print(p.pop());  
    }  
}
```

p =



x = 2

Affichage :

> 2 5 4 3 1

Exemple

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    ► while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```

p =



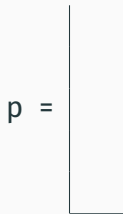
x = 2

Affichage :

> 2 5 4 3 1

Example

```
public static void main (String[] args) {  
    Stack<Integer> p = new Stack<Integer>();  
    p.push(1);  
    p.push(2);  
    int x = p.pop();  
    System.out.print(x);  
    p.push(3);  
    p.push(4);  
    p.push(5);  
    while (!p.empty()) {  
        System.out.print(p.pop());  
    }  
}
```



x = 2

Affichage :

> 2 5 4 3 1

Piles vs Tableaux

Les **piles** et les **tableaux** sont deux structures de données qui servent à stocker des éléments. Quelles différences?

Piles (exemple : `Stack<Integer>`)

Tableaux (exemple : `int[]`)

Piles vs Tableaux

Les **piles** et les **tableaux** sont deux structures de données qui servent à stocker des éléments. Quelles différences?

Piles (exemple : `Stack<Integer>`)

- ▶ Ne peut stocker que des types références.

Tableaux (exemple : `int[]`)

- ▶ Peut stocker n'importe quel type de données.

Piles vs Tableaux

Les **piles** et les **tableaux** sont deux structures de données qui servent à stocker des éléments. Quelles différences?

Piles (exemple : `Stack<Integer>`)

- ▶ Ne peut stocker que des types références.
- ▶ Taille variable et illimitée.

Tableaux (exemple : `int[]`)

- ▶ Peut stocker n'importe quel type de données.
- ▶ Taille finie, fixée au départ.

Piles vs Tableaux

Les **piles** et les **tableaux** sont deux structures de données qui servent à stocker des éléments. Quelles différences?

Piles (exemple : `Stack<Integer>`)

- ▶ Ne peut stocker que des types références.
- ▶ Taille variable et illimitée.
- ▶ Ne peut ajouter ou retirer des éléments que sur le dessus.

Tableaux (exemple : `int[]`)

- ▶ Peut stocker n'importe quel type de données.
- ▶ Taille finie, fixée au départ.
- ▶ Peut accéder à tous les éléments sans restriction.

Cas d'utilisation : analyse syntaxique

Expressions arithmétiques

Une **expression arithmétique**, c'est un calcul avec des nombres, des opérations (+, −, ×, /) et des parenthèses :

$$(3 + 2) \times (7 - (2 + 2))$$

Expressions arithmétiques

Une **expression arithmétique**, c'est un calcul avec des nombres, des opérations (+, −, ×, /) et des parenthèses :

$$(3 + 2) \times (7 - (2 + 2))$$

Défi : écrire une fonction Java `evaluate` qui prend en entrée une chaîne de caractères représentant une expression arithmétique, et renvoie le résultat du calcul. Par exemple, `evaluate("(3+2)*(7-(2+2))")` doit renvoyer 15.

Expressions arithmétiques

Une **expression arithmétique**, c'est un calcul avec des nombres, des opérations (+, −, ×, /) et des parenthèses :

$$(3 + 2) \times (7 - (2 + 2))$$

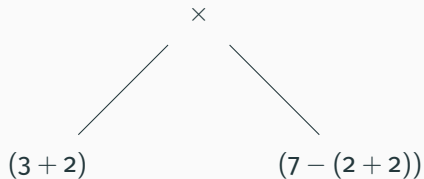
Défi : écrire une fonction Java `evaluate` qui prend en entrée une chaîne de caractères représentant une expression arithmétique, et renvoie le résultat du calcul. Par exemple, `evaluate("(3+2)*(7-(2+2))")` doit renvoyer 15.

Attention : c'est pas facile du tout ! La structure de **pile** va être utile.

La notation avec des parenthèses cache une structure d'arbre :

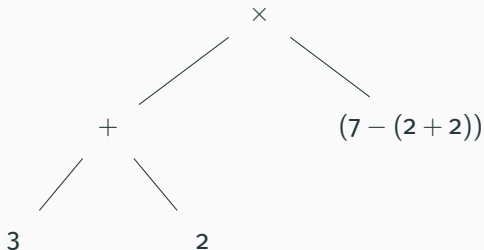
$$(3 + 2) \times (7 - (2 + 2))$$

La notation avec des parenthèses cache une structure d'arbre :



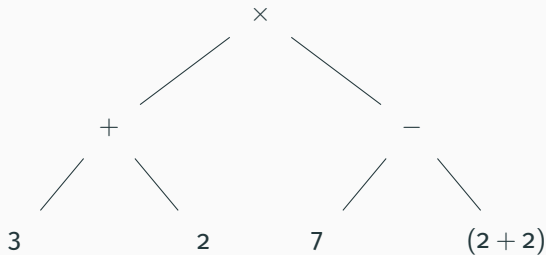
Arbre syntaxique

La notation avec des parenthèses cache une structure d'arbre :



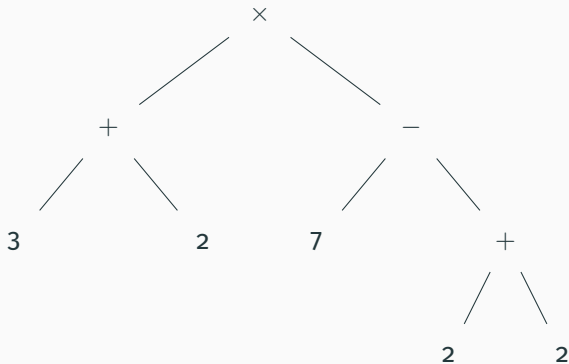
Arbre syntaxique

La notation avec des parenthèses cache une structure d'arbre :



Arbre syntaxique

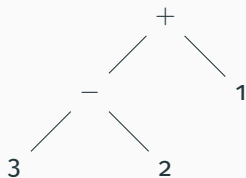
La notation avec des parenthèses cache une structure d'arbre :



Le rôle des parenthèses

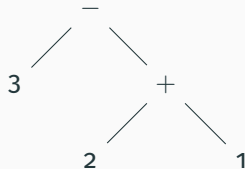
Les parenthèses servent à éviter les ambiguïtés, par exemple :

$$(3 - 2) + 1$$



vs

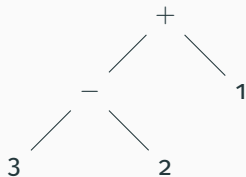
$$3 - (2 + 1)$$



Le rôle des parenthèses

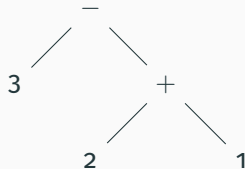
Les parenthèses servent à éviter les ambiguïtés, par exemple :

$$(3 - 2) + 1$$



vs

$$3 - (2 + 1)$$



... et si il existait un autre moyen ?

Notation polonaise et notation polonaise inverse

Plusieurs façons d'écrire les expressions arithmétiques :

- ▶ $x + y$: forme **infixe**.
- ▶ $+xy$: forme **préfixe** (ou **notation polonaise**).
- ▶ $xy+$: forme **postfixe** (ou **notation polonaise inverse**).

Notation polonaise et notation polonaise inverse

Plusieurs façons d'écrire les expressions arithmétiques :

- ▶ $x + y$: forme **infixe**.
- ▶ $+xy$: forme **préfixe** (ou **notation polonaise**).
- ▶ $xy+$: forme **postfixe** (ou **notation polonaise inverse**).

Notation inventée en 1924 par Jan Łukasiewicz.

https://fr.wikipedia.org/wiki/Notation_polonaise_inverse



Calculatrice Friden EC-132 (1964)

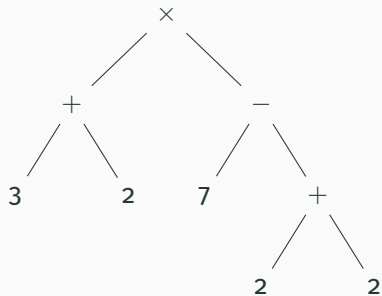
Conversion : arbre vers forme préfixe

forme infixe :

$$(3 + 2) \times (7 - (2 + 2))$$

forme préfixe :

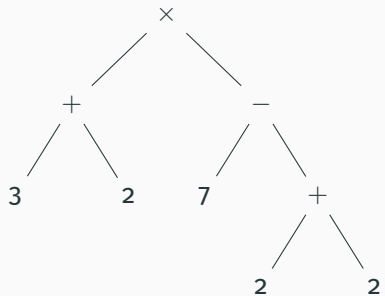
$$\times + 3 2 - 7 + 2 2$$



Conversion : arbre vers forme préfixe

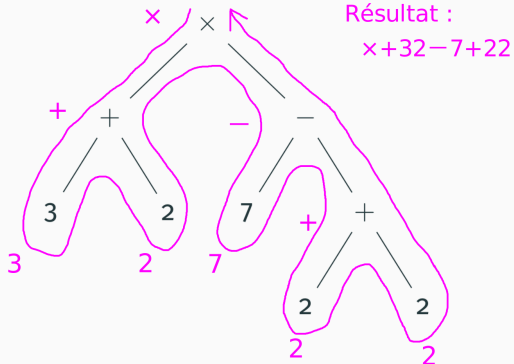
forme infixe :

$$(3 + 2) \times (7 - (2 + 2))$$



forme préfixe :

$$\times + 3 2 - 7 + 2 2$$



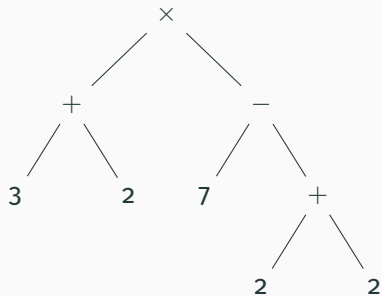
Conversion : arbre vers forme postfixe

forme infixe :

$(3 + 2) \times (7 - (2 + 2))$

forme postfixe :

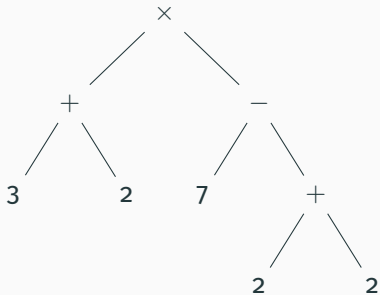
$3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$



Conversion : arbre vers forme postfixe

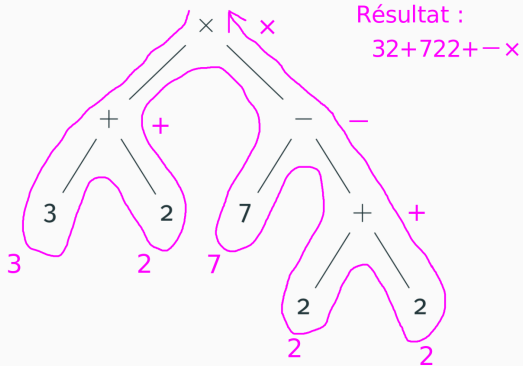
forme infixe :

$$(3 + 2) \times (7 - (2 + 2))$$



forme postfixe :

$$3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$$



Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$



Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 2 + - ×



Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : $3\ 2 + 7\ 2\ 2 + - \times$



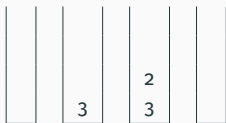
Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$



Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$



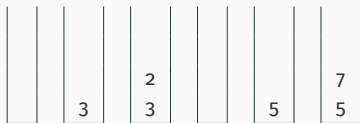
Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 2 + - ×



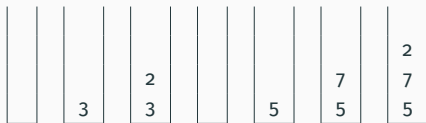
Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 + - ×



Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 + - ×

							2
		2			7	2	2
	3	3		5	5	7	7
					5	5	5

Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 2 + - ×

							2	
		2			7	2	2	7
	3	3		5	5	5	5	5

Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 2 + - ×

							2		
		2			7	2	2		4
	3	3		5	5	7	7	7	7
					5	5	5	5	5

Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 2 + - ×

							2			
		2			7	2	2		4	
	3	3		5	5	7	7	7	7	
						5	5	5	5	5

Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 2 + - ×

							2				
		2			7	2	2		4		
	3	3		5	5	7	7	7	7		3
						5	5	5	5	5	5

Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 2 + - ✗

							2					
		2			7	2	2		4		3	
	3	3		5	5	5	7	5	7	5	5	

Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérande, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : 3 2 + 7 2 2 + - ✕

							2						
		2			7	2	2		4				
	3	3		5	5	7	7	7	7		3		
						5	5	5	5	5	5		15

Évaluer une expression sous forme postfixe

Algorithme

On parcourt l'expression de gauche à droite

- ▶ Si l'élément rencontré est un opérateur, on l'empile
- ▶ Sinon (c'est un opérateur) :
 - On dépile deux éléments
 - On exécute l'opération et on empile son résultat
- ▶ À la fin, la pile contient le résultat de l'évaluation de l'expression

forme postfixe : $3\ 2\ +\ 7\ 2\ 2\ +\ -\ \times$

							2						
		2			7	2	2		4				
	3	3		5	5	7	7	7	7		3		
						5	5	5	5	5	5		15

```
public static int evaluatePostfix(String[] expr) {
    Stack<Integer> p = new Stack<Integer>();
    for (String s : expr) {
        if (s.equals("+") || s.equals("-") || s.equals("*") || s.equals("/")) {
            int x = p.pop();
            int y = p.pop();
            switch (s) {
                case "+": p.push(y + x); break;
                case "-": p.push(y - x); break;
                case "*": p.push(y * x); break;
                case "/": p.push(y / x); break;
            }
        } else { // Sinon : c'est un operande
            p.push(Integer.valueOf(s));
        }
    }
    return p.pop();
}
```